

How RAG Works: Tokens, Embeddings, MongoDB & Qdrant

Concrete Technical Deep Dive using "The Wonderful Wizard of Oz" by L. Frank Baum

Step 1: Document Text & Tokenization

☒ "The Wonderful Wizard of Oz" (Chapter 1 Excerpt):
"Dorothy lived in the midst of the great Kansas prairies, with Uncle Henry, who was a farmer, and Aunt Em, who was the farmer's wife. Their house was small..."

Raw String: 198 characters • 34 words • UTF-8 Byte Stream

↓ Byte-Pair Encoding (BPE) Tokenizer ↓

Token Sub-Word Segmentation (Vocabulary IDs):

Dorothy
#14820

lived
#5829

in
#287

the
#262

midst
#27192

of
#286

the
#262

great
#1049

Kansas
#14902

prairies
#39108

farmer
#8912

... (42 tokens)

• 1 token ≈ 4 chars • Tokens map words to integer vectors

Step 2: Dense Semantic Embeddings

Embedding Model: nomic-embed-text (or text-embedding-004)
Transforms token sequence into a normalized 768-dim float32 vector:

$$\mathbf{v} = [0.0412, -0.0891, 0.1205, \dots, 0.0074] \in \mathbb{R}^{768}$$

Normalized length: $||\mathbf{v}|| = 1.0$ (Unit hypersphere for cosine similarity)

Semantic Hyperspace (Geometric Distance = Meaning Similarity):

Dimension 2 (Characters / Roles)

v_chunk1
"Dorothy lived in the great Kansas prairies..."

v_query
"Where did Dorothy live? Emerald City gates..."
(cos θ = 0.310 - Low Relevance)

$$\text{Cosine Similarity: } \cos(\theta) = (\mathbf{u} \cdot \mathbf{v}) / (||\mathbf{u}|| ||\mathbf{v}||) \in [-1.0, +1.0]$$

Step 3: Dual Database Architecture

☒ MongoDB 7.0 (Relational / Document DB)
Collection: "documents" • Full Record & Version History

```
{
  "_id": ObjectId("66fc10..."),
  "title": "The Wonderful Wizard of Oz",
  "bibtext": { "author": "Baum, L. Frank", "year": "1900" },
  "fullText": "Dorothy lived in the midst of...",
  "version": 1, "summaries": { "llama": {...}, "mistral": {...} }
}
```

✂ Qdrant Vector DB (HNSW Index Engine)
Collection: "document_chunks" • Sub-millisecond ANN

```
{
  "id": "c1b48f92-7f31-4a9c-9c71-09ba192f1b0a",
  "vector": [ 0.0412, -0.0891, 0.1205, ... ],
  "payload": {
    "documentId": "doc-oz-1900", "chunkIndex": 0,
    "text": "Dorothy lived in the midst of the great Kansas prairies..."
  }
}
```

Step 4: Interactive RAG Query Execution Pipeline (Retrieval → Augmentation → Generation)

☒ 1. User Research Question
"Where did Dorothy live and what was her home like?"
• Query Tokens: 11 tokens
• Model: nomic-embed-text
• Vector: $\mathbf{v_query} \in \mathbb{R}^{768}$

➡

✂ 2. Qdrant HNSW Search
Filter: documentId == "doc-oz-1900"
Metric: Cosine Similarity • Top-K = 1

☒ Match: Chunk #0 (Score: 0.892)
"Dorothy lived in the midst of the great Kansas prairies... house was small"
Relevance: 89.2% • Latency: 2.1 ms

➡

☒ 3. Prompt Augmentation

[SYSTEM: Answer based strictly on context]
[CONTEXT CHUNK 0 (Relevance: 89%)]:
"Dorothy lived in Kansas prairies... four walls, a floor and a roof, one room..."
[USER QUESTION]:
"Where did Dorothy live and what was her home like?"

➡

☒ 4. Llama 3.3 (70B) Inference

Grounded Answer Output:
"Dorothy lived in the midst of the Kansas prairies with Uncle Henry and Aunt Em. Her home was a small, one-room wooden house..."
☒ Citation: [Chunk 0 • Kansas Prairies]

Core Architectural Distinctions: MongoDB (Document Store) vs. Qdrant (Vector Database)

CAPABILITY / ROLE	MONGODB 7.0 (DOCUMENT REPOSITORY)	QDRANT (VECTOR RETRIEVAL ENGINE)
Primary Data Unit	Hierarchical BSON documents (full text, metadata, authors, version history)	High-dimensional points (vector float32 arrays + small payload metadata)
Query Mechanism	B-Tree index lookups, exact key matches, regex, aggregation pipelines	Approximate Nearest Neighbor (ANN) search via HNSW graphs (Cosine / Dot product)
RAG System Purpose	Single source of truth for raw assets, BibTeX fields, versions, and generated summaries	Sub-second semantic passage locator to extract exact relevant context for prompt injection